Design Document

Edge Coffee Machine

Living document · Snapshot: 2025-12-21

For the most up-to-date version, click here.

Piervito Creanza
Matteo Delton
Emiliano Finetti
Maxime Trimboli
Javier Asensio Castillo
Jorge Jiménez Oropesa
Balša Rakočević

POLITECNICO
MILANO 1863

Mälardalen
University

Contents

1 Introduction & Background

1.1 Document Purpose

The purpose of this Design Document is to detail the architectural and technical specifications for the Edge Coffee Machine. Building on the Requirements Analysis and Specification Document (RASD), this document serves as a comprehensive guide for the system's implementation. It aims to assist developers, architects, and other stakeholders in understanding the structural design, component interactions, and technical decisions that will define the Edge Coffee Machine.

1.2 Background

1.2.1 Team & Collaboration

The ECM is an academic group project for the Distributed Software Development course, emphasizing international development collaboration. The development team includes a total of 7 students, including 4 students from the Politecnico of Milano (PoliMi, Italy) and 3 students from Mälardalen University (MDU, Sweden). This team is also closely supported by one coordinator from each university.

1.2.2 Customer

The customer for this project is the Qt Group, a software company primarily known for its open-source User Interface (UI) frameworks. The company provides products like Qt for MCU, which enables the creation of "smartphone-like user experiences" on resource-constrained embedded devices.

1.2.3 Objectives

The ECM project focuses on demonstrating the capabilities of Edge AI on microcontrollers by creating an AI-driven coffee machine application. However, our objective is not only to showcase the technical power of Edge AI; we also aim to illustrate how technology can anticipate and respond to human needs in everyday moments. The system will recognize each user, remember their preferences, and offer a personalized coffee experience that feels natural and effortless. By running AI directly on the device, the Edge Coffee Machine ensures instant responses, enhanced privacy, and energy-efficient performance.

1.3 Project Scope

This design document covers the development of a fully functional application for the Edge Coffee Machine. The scope encompasses:

- User Interface: Creating a fluid, high-performance interface that handles all user interactions (touchscreen, camera image recognition, and voice commands).
- Application Logic: Implementing the core business logic: mainly customer classification and personalized drink suggestions.
- AI Component: Integrating AI models for image recognition and voice commands on the target microcontroller.

The integration with the physical coffee machine hardware and the handling of payments are out of scope.

2 High-Level System Description

At a high level, the Edge Coffee Machine is designed as an autonomous embedded system that recognizes users and tailors its interface and functionalities to their preferences. The system consists of three main layers: Input Layer, Processing Layer, and Output Layer.

2.1 Input Layer

The system collects data from multiple input sources:

- **Camera**: Captures an image of the user to perform facial recognition.
- **Microphone**: Captures predefined voice commands to control menu navigation and drink selection.
- **Touchscreen Display**: Allows manual user input for customization or confirmation of selections.

2.2 System Interfaces

This section defines the external interfaces of the system that enable interaction with users and hardware components.

2.2.1 User Interfaces

- *Touch Interface* (Qt for MCU): Primary means of interaction for drink customization and confirmation.
- *Voice Interface*: Allows users to issue specific commands such as "make espresso" or "show menu."
- *Vision Interface* (Camera): Detects and recognizes users automatically without manual input.

2.2.2 Software Interfaces

- *Frontend (Qt for MCU)*: Responsible for rendering the GUI and handling user input.
- *Backend (ESP-IDF in C++)*: Contains the core application logic, communication with the hardware peripherals, and management of AI inference results.
- *AI Runtime Libraries (ESP-DL or emlearn)*: Used to execute trained machine learning models for facial and voice recognition.

- *Internal Communication Protocols*: Communication between modules (e.g., between backend and AI runtime) is handled via in-memory message passing within the microcontroller's environment.

2.2.3 Hardware Interfaces

The ESP32-P4-Function-EV microcontroller connects directly to:

- Camera module (for vision input)
- Microphone (for audio input)
- Touch display (for interaction and visual output)

All components are integrated into a single development board platform, eliminating the need for external hardware dependencies.

2.3 Hardware Overview

The ESP32-P4-Function-EV microcontroller is the core of the system. It is equipped with:

- A dual-core processor optimized for parallel multimedia tasks.
- A Neural Processing Unit (NPU) to accelerate AI inference.
- A 7" capacitive touchscreen display.
- A camera for face detection and recognition.
- A microphone for capturing voice commands.
- Multiple I/O ports for possible future extensions.

The use of this hardware enables real-time AI execution at the edge, supporting the project's primary design principle: intelligent interaction without external dependency.

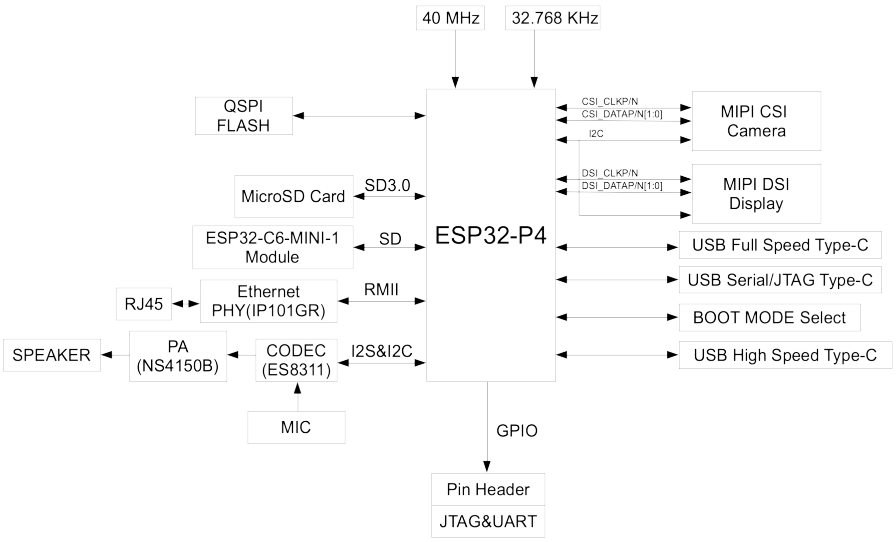


Figure 1: ESP32-P4-Function-EV board block diagram.

3 Architectural Design

3.1 Overview

This chapter details the high-level software architecture for the ECM. It defines system design principles, including the Modular Monolith structure and the Edge AI approach.
The architecture is broken down into its primary logical components (*UI/Frontend*, *App logic/Backend* and *AI component*) which are needed to fulfill the system requirements, as specified in the RASD, and they will be described in detail.

3.2 Component View

This section defines the static architecture of the system, decomposing it into its main sub-modules. For each component, its responsibilities and the interfaces it uses to interact with other modules are described.

The system is decomposed into three primary software components, as illustrated in Figure 2. All modules are compiled into a single firmware and run on the device.

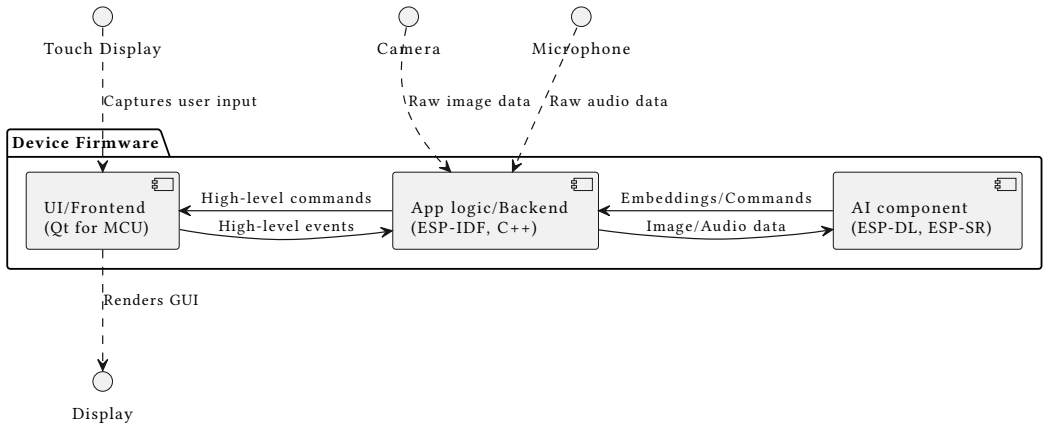


Figure 2: Component View

- **UI/Frontend (Qt for MCU):**
  This component is responsible for rendering the graphical user interface (GUI) on the *Display* and processing user input from the *Touch Display* and it is implemented using Qt for MCU.

  It receives high-level commands from the *Backend* and draws the corresponding screen on the *Display*, and sends to the *Backend* high-

level events after translating them from raw touch data obtained from the *Touch Display*.

- **App logic/Backend (ESP-IDF):**
  This is the core "brain" of the application. It is written in C++ and contains all business logic. While the final target platform will run on the ESP-IDF framework, the component is developed and tested in a desktop environment using the Qt framework to ensure logic and interfaces are correct before hardware integration. It orchestrates the other modules and decides what actions to take:
  - It directly processes the high-level events arriving from the *UI*, giving back a high-level command to update it.
  - It receives raw data from the *Camera* and passes them to the *AI component*, from which it obtains the embedding of the detected customer.
  - It receives raw data from the *Microphone* and passes them to the *AI component*, from which it obtains the command to perform.
- **AI component (ESP-DL, ESP-SR):**
  This specialized module is responsible for executing the pre-trained machine learning models.

  It receives data (images or audio) from the *Backend*, performs inference using ESP-DL and/or ESP-SR runtime, and returns a simple result (e.g., customer embedding or command) to the *Backend*.

3.3 Data Flow Diagram

The following diagram illustrates the data flow between the different layers of the system.

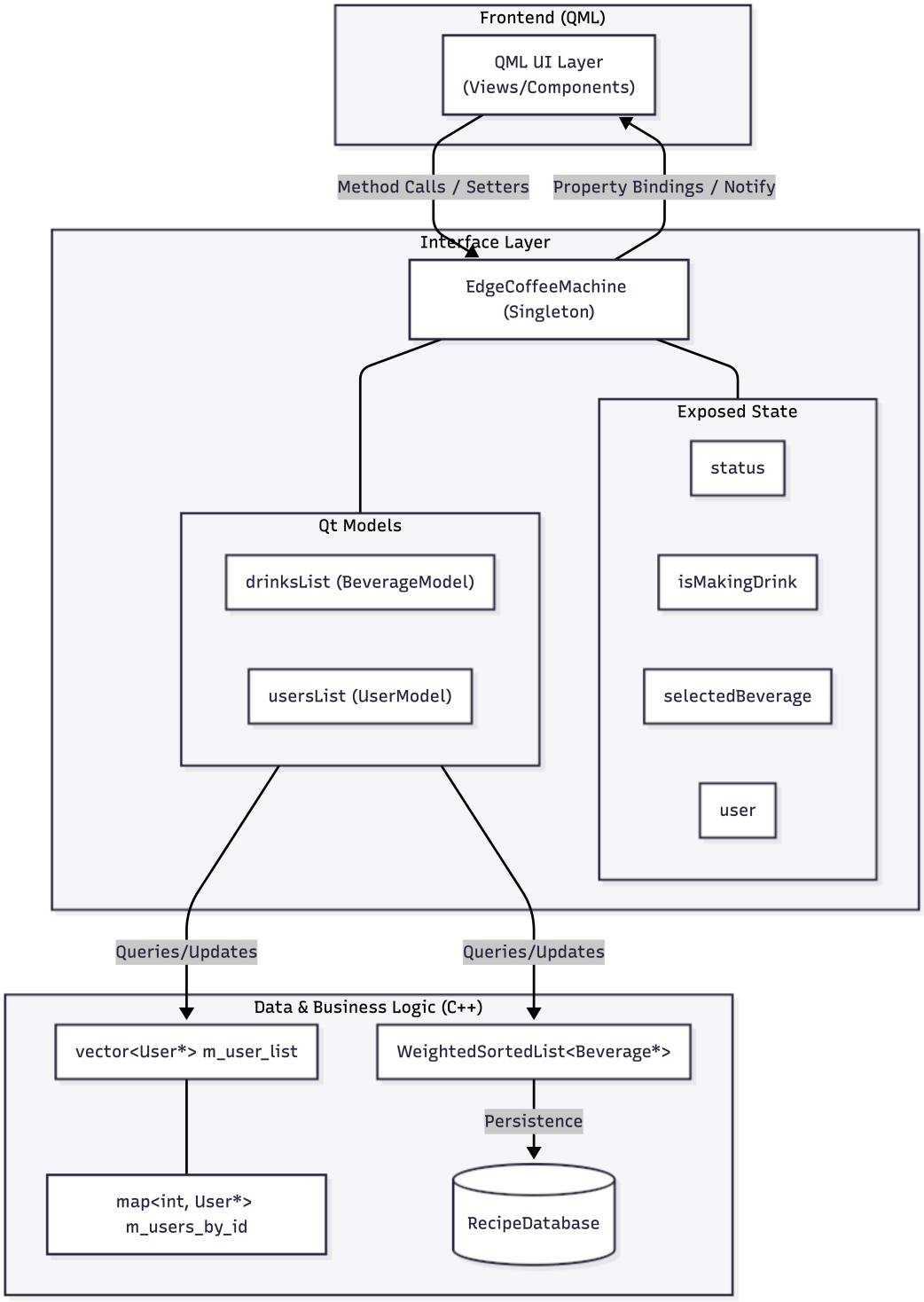


Figure 3: Data Flow Diagram

3.4 Deployment View

The deployment architecture for this project is a single-node embedded system. All software modules (*Frontend*, *Backend*, and *AI*) are compiled

together into a **single monolithic application firmware**. This single firmware binary is then "flashed" onto and executed entirely by the **ESP32-P4-Function-EV** hardware.

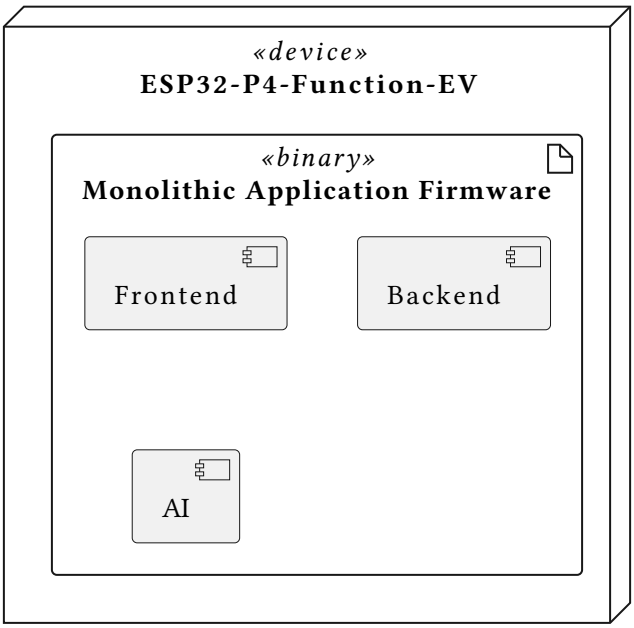


Figure 4: Deployment View

This single-node approach contrasts with typical client-server or cloud-based architectures. By embedding all logic directly into the device, the system guarantees full offline functionality, minimizes latency for real-time interactions (like facial recognition), and ensures user data never leaves the device, providing a strong foundation for privacy.

3.5 Runtime View

In this section, we will illustrate the principal sequences of interactions between the components of the system to accomplish the main tasks of the platform.

3.5.1 User Recognition and Menu Display

The *User* approaches the ECM. The *Backend* continuously receives images from the *Camera* and sends them to the *AI component*.

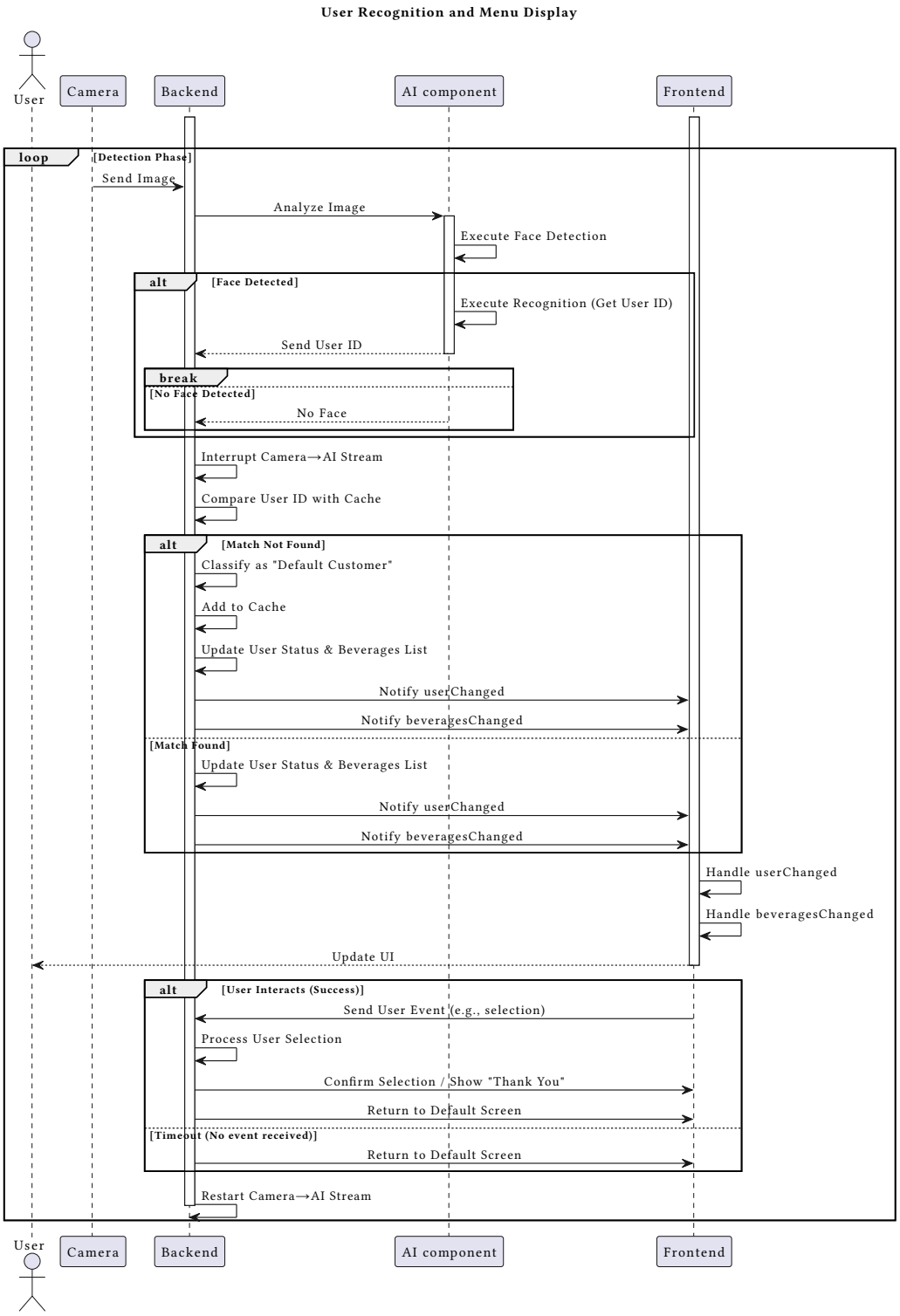
The latter executes a detection model to verify that a face is present in the image and, if it is, where it is located. If and only if a face is detected, the

AI component executes another model to effectively recognize the customer and obtain their User ID.

After that, the User ID is sent to the *Backend*, which interrupts the data flow between the *Camera* and the *AI component*. The *Backend* compares the User ID received with those present in memory (cache), each of which is associated with a customer already recognized and served.

If a match is found, the *Backend* updates its internal state by setting the recognized user as the current one. If no match is found, it sets the current user to a guest state (`null`). In both cases, it emits signals (`userChanged`, `beveragesChanged`) to notify the *Frontend* that its state has changed. The *Frontend* reacts to these signals by reading the updated properties (the current user and the corresponding beverage list) and refreshing the display to show either a personalized or a guest view.

The *Backend* waits for events coming from the *Frontend*; if it does not receive them after a certain delay, it communicates to the *Frontend* to return to the default screen and restarts the data flow between the *Camera* and the *AI component*.



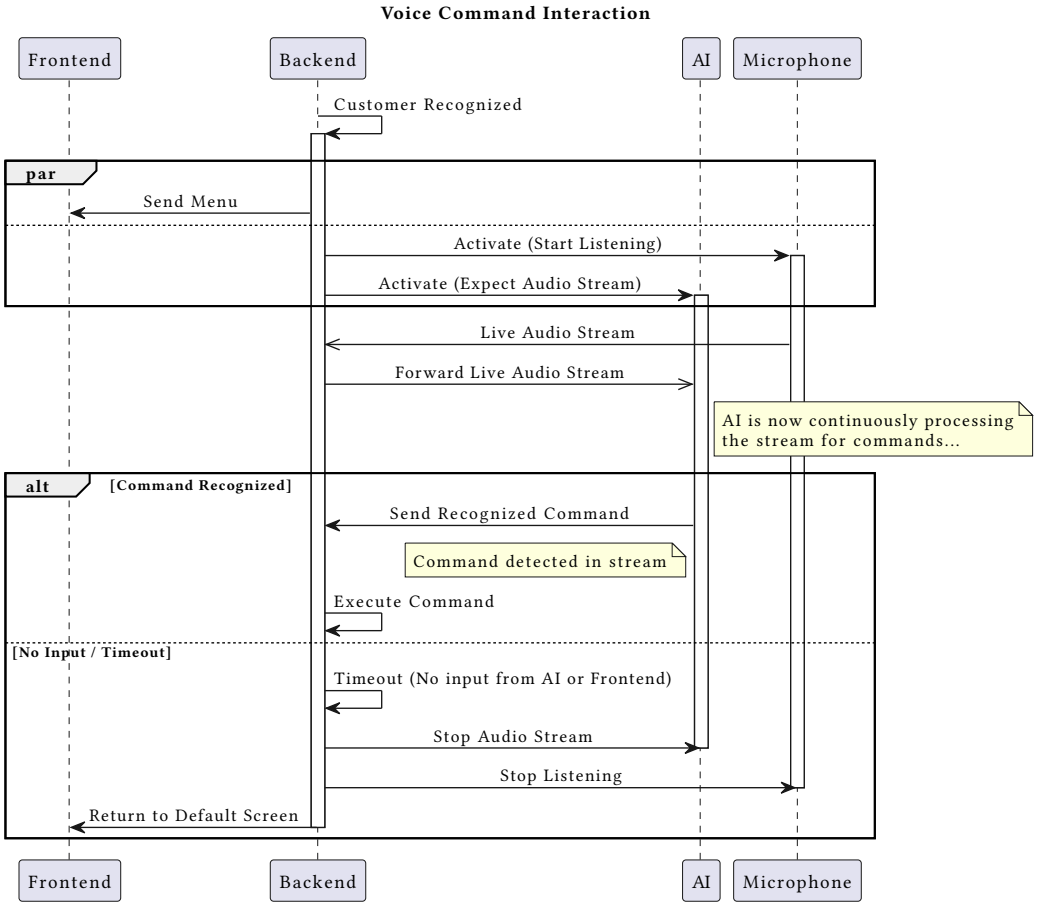
3.5.2 Voice Command Interaction

Immediately after customer recognition, as the Backend sends the menu to the Frontend, it also initiates the voice interaction in parallel.

The *Backend* begins listening to the *Microphone* and starts streaming the resulting audio data directly to the *AI component*. The *AI component* activates and continuously executes a model to detect and recognize voice commands within this live stream.

If a command is recognized, the *AI component* sends it to the *Backend*, which executes it.

If the *Backend* doesn't receive any input from either the *AI component* or the *Frontend*, it stops the audio stream from the *Microphone* to the *AI component* and communicates to the *Frontend* to return to the default screen.



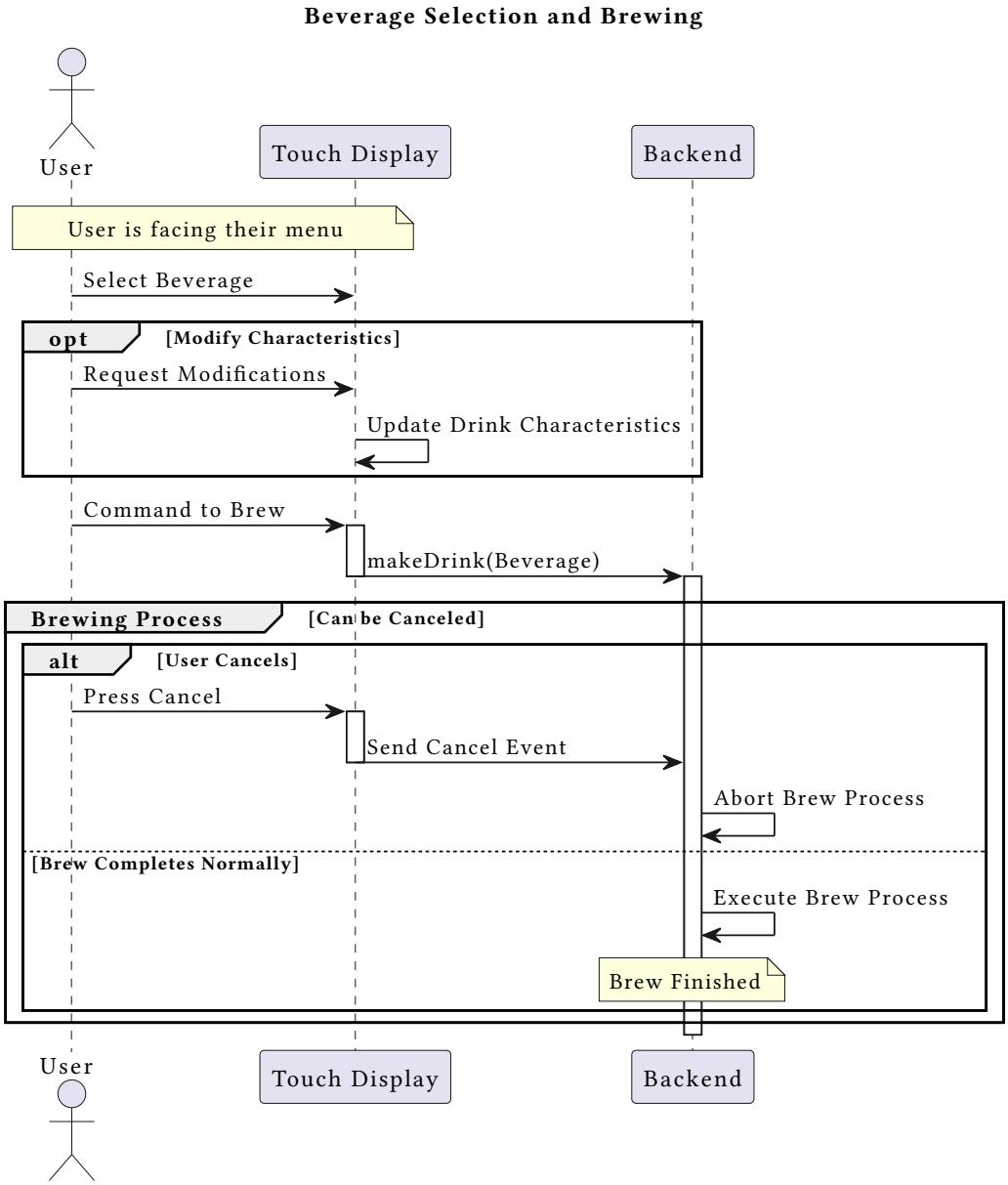
3.5.3 Drink Customization Through Touch Display

After the customer recognition process is complete and the customer is viewing their menu, they interact with the *Touch Display* to select one of the proposed beverages.

After the selection, they can request a modification of one or more characteristics of the selected drink.

When they finally give the command to brew their choice, the *Frontend* calls the `makeDrink` method on the *Backend*, passing the selected `Beverage` object as a parameter..

The latter now begins the process to brew the selected beverage. During this brewing process, the user has the option to cancel the order via the *Touch Display*. If a cancel command is given, the *Frontend* sends a "cancel event" to the *Backend*, which then aborts the brewing process.



3.6 Component Interfaces

This section defines the formal API and data contracts between the C++ *Backend* and the QML *Frontend*. The communication is based on the Qt for MCUs property system (`Qul::Property`), which ensures a clean decoupling between logic and presentation while enabling reactive UI updates.

The main elements of the API are:

- **C++ Objects**: Strongly-typed C++ objects exposed to QML (`EdgeCoffeeMachine`, `Beverage`, `User`, `IngredientInfo`)

- **Properties**: Reactive state exposed by the *Backend* via `Qul::Property`
- **Methods**: C++ functions callable from QML to trigger actions
- **Signals**: Model change notifications (`modelReset()`) for list updates

3.6.1 UI/Frontend

The *Frontend* is responsible for rendering the UI based on the state exposed by the *Backend*'s properties. It triggers actions by calling the *Backend*'s methods in response to user input (e.g., button presses). The *Frontend* automatically reacts to property changes through QML property bindings.

3.6.2 App logic/Backend

The *Backend* is implemented as a singleton class `EdgeCoffeeMachine` that serves as the central controller. It exposes all state and functionality to QML through a well-defined interface.

3.6.2.1 `EdgeCoffeeMachine`

Properties

- `Qul::Property<std::string> status`:
  Human-readable status of the machine (e.g., "Idle", "Making Espresso…", "Espresso is ready!", "Welcome `UserName`"). The UI binds to this property and automatically updates when the value changes.
- `Qul::Property<bool> isMakingDrink`:
  Boolean flag that is `true` during the brewing process. The UI uses this to show/hide the brewing screen, display progress animations, and disable other interactions during brewing.
- `Qul::Property<Beverage*> selectedBeverage`:
  Pointer to the currently selected beverage object. The UI binds to this to display the featured drink's details (name, ingredients, image). Can be `nullptr` if no selection has been made.
- `Qul::Property<User*> user`:
  Pointer to the currently logged-in user. It is `nullptr` when in "Guest Mode". The UI binds to this property to switch between guest and personalized views, displaying the user's name in the welcome message.
- `Qul::Property<BeverageModel*> drinksList`:
  Pointer to the BeverageModel instance that exposes the beverage list to

QML. This model dynamically switches between the global popularity list (guest mode) and the user's personalized list (logged-in mode).

- `Qul::Property<UserModel*> usersList`:
  Pointer to the UserModel instance that exposes the list of all registered users. Used by the manual user selection screen.

Methods

- `void makeDrink(Beverage* beverage)`:
  Initiates the brewing process for the specified beverage. If `beverage` is `nullptr`, uses the `selectedBeverage`. Validates machine state (must be idle), calculates brewing time based on ingredients, sets `isMakingDrink` to `true`, updates `status`, starts the internal brew timer, and automatically logs out the user when brewing completes (via `finishBrewing()`).
- `void stopBrewing()`:
  Immediately cancels the active brewing process. Stops the brew timer, sets `isMakingDrink` to `false`, and updates the status message. Called when the user taps the cancel button during brewing.
- `void selectBeverage(Beverage* beverage)`:
  Sets the currently selected beverage. Updates the `selectedBeverage` property and the status message (e.g., "Selected: Cappuccino"). For guest users, automatically resets ingredients to defaults.
- `void setUser(User* user)`:
  Logs in or logs out a user. When `user` is not `nullptr`, logs in the specified user, updates the welcome message in `status`, and switches `drinksList` to the user's personalized beverage list. When `user` is `nullptr`, logs out the current user and switches back to the global popularity list.
- `void enrollUser(int id)`:
  Registers a new user with the specified AI-detected ID. Creates a new `User` object with default settings, adds it to the internal user registry with fast lookup by ID, updates the `usersList` model, and automatically logs in the new user via `setUser()`.
- `void identifyUser(int id)`:
  Identifies and logs in an existing user by their AI-detected ID. Looks

up the user in the internal registry and calls `setUser()` if found. Logs a warning if the ID is not found.

- `void logoutUser()`:
  Logs out the current user by calling `setUser(nullptr)`. Returns the machine to guest mode.
- `Beverage* getSelectedBeverage() const`:
  Getter method that returns the currently selected beverage pointer.
- `const std::vector<Beverage*>& getPopularBeverages() const`:
  Returns the global list of beverages sorted by popularity. Used internally and by the User class for initialization.
- `bool getIsMakingDrink() const`:
  Getter method that returns the brewing state.
- `User* getUser() const`:
  Getter method that returns the current user pointer.

3.6.2.2 `Beverage`

The `Beverage` class represents a drink configuration and is exposed to QML as a data object.

Properties

- `Qul::Property<std::string> name`: Display name of the beverage
- `Qul::Property<IngredientInfo*> coffeeBeans`: Coffee ingredient configuration
- `Qul::Property<IngredientInfo*> cocoaPowder`: Cocoa ingredient configuration
- `Qul::Property<IngredientInfo*> water`: Water ingredient configuration
- `Qul::Property<IngredientInfo*> foam`: Foam ingredient configuration
- `Qul::Property<IngredientInfo*> milk`: Milk ingredient configuration

Methods

- `void resetIngredients()`: Resets all ingredients to their default values. Called from QML when the user taps the reset button in the settings panel.
- `int brewingTime()`: Calculates and returns the estimated brewing time in milliseconds based on current ingredient quantities.

3.6.2.3 `User`

The `User` class represents a user profile with personalized preferences.

Properties

- `Qul::Property<std::string> name`: User's display name
- `Qul::Property<std::string> initials`: User's initials (derived from name)
- `Qul::Property<int> picture`: Index of the user's profile picture

Methods

- `void beverageBrewed(Beverage* beverage)`: Increments the selection counter, updates the recommendation model weights, adjusts user classification (Default/Conservative/Early Adopter), and refreshes the personalized beverage list.
- `void beverageCustomized()`: Flags the current session as "customized" to boost the `customizerScore` during the next brewing event.
- `const std::vector<Beverage*>& getDisplayBeverages() const`: Returns the user's personalized beverage list, sorted according to their classification category.

3.6.2.4 `IngredientInfo`

The `IngredientInfo` struct defines the configuration for a single ingredient.

Properties

- `Qul::Property<float> current`: Current amount (in absolute physical units: grams or milliliters)
- `Qul::Property<float> min`: Minimum allowed amount (absolute units)
- `Qul::Property<float> max`: Maximum allowed amount (absolute units)
- `Qul::Property<float> def`: Default amount (absolute units)

3.6.2.5 List Models (`BeverageModel` & `UserModel`)

Both models inherit from `Qul::ListModel` and provide a reactive interface for QML ListView components.

`BeverageModel`:

- `int count() const`: Returns the number of beverages in the list
- `Beverage* data(int index) const`: Returns the beverage at the specified index

- `void updateList(const std::vector<Beverage*>& newList)`: Updates the model with a new list and emits `modelReset()` to trigger UI refresh

`UserModel`:

- `int count() const`: Returns the number of registered users
- `User* data(int index) const`: Returns the user at the specified index
- `void updateList(const std::vector<User*>& newList)`: Updates the model with a new list and emits `modelReset()` to trigger UI refresh

3.6.2.6 Signals and Reactive Updates

The architecture uses the Qt for MCUs property system for reactive UI updates:

- **Property Bindings**: When QML binds to a `Qul::Property` (e.g., `EdgeCoffeeMachine.status`), the UI automatically updates whenever the C++ code calls `setValue()` on that property.
- **Model Signals**: When `BeverageModel::updateList()` or `UserModel::updateList()` is called, the model emits `modelReset()`, which forces the QML ListView to completely redraw with the new data.
- **Timer Callbacks**: The internal brew timer uses a callback lambda (`m_brewTimer.onTimeout([this]() { finishBrewing(); })`) that is not exposed to QML. When the timer expires, it calls `finishBrewing()`, which updates properties that the UI is bound to.

3.6.2.7 State Diagram: Drink Lifecycle

The following PlantUML diagram illustrates the complete lifecycle of a drink order, from idle state through selection, customization, brewing, and completion:

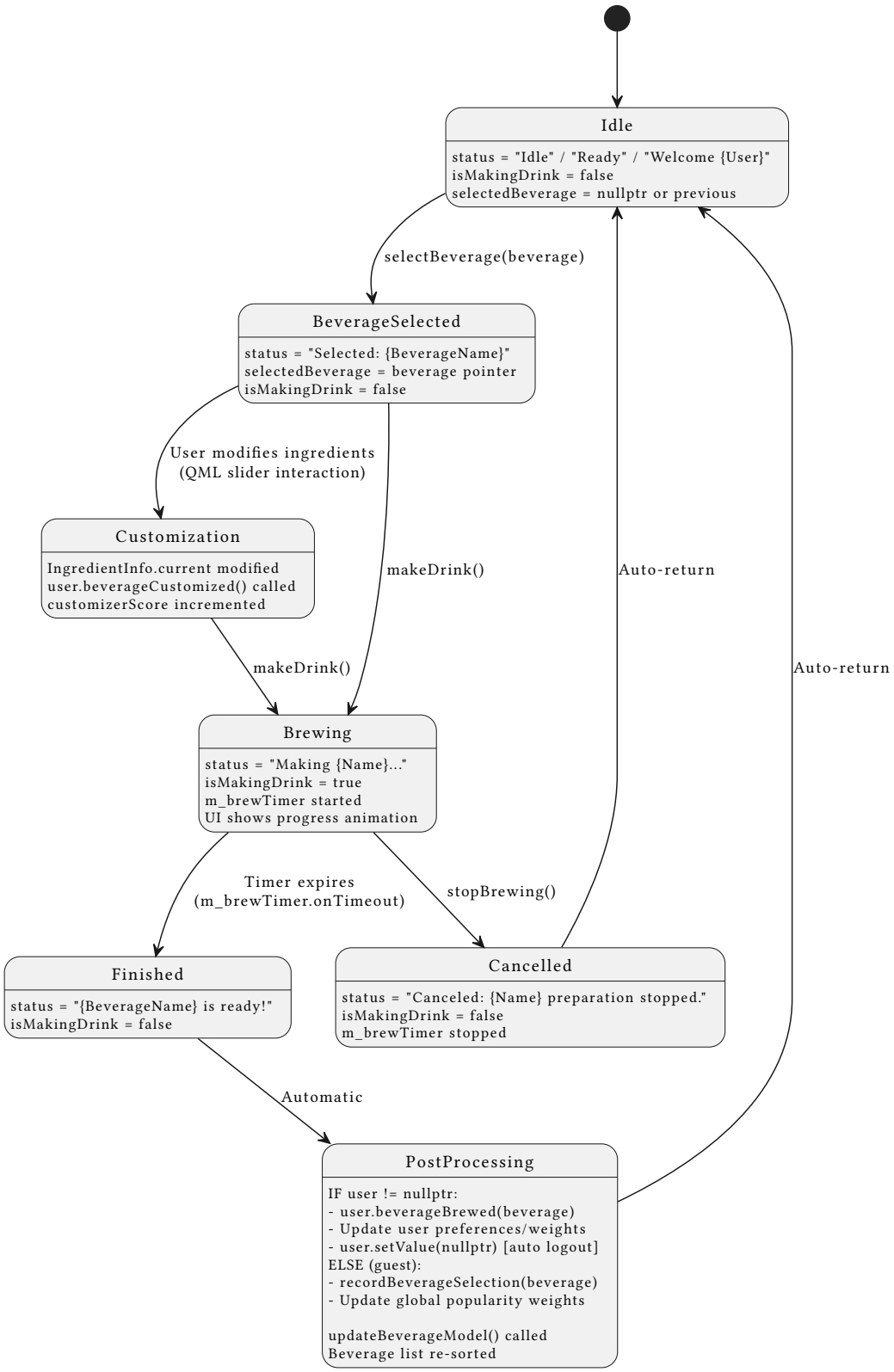


Figure 8: Drink Lifecycle State Diagram

3.6.2.8 QML Binding Examples

The following real snippets from the QML layer show how the UI binds to backend properties and methods.

Property binding (automatic updates)

```
// Home.qml – featured card updates when the selection
changes
Components.FavouritePanel {
  id: favouritePanel
  targetBeverage: EdgeCoffeeMachine.selectedBeverage
}
```

List model binding and selection

```
// components/DrinkListPanel.qml – ListView bound to
drinksList model
ListView {
  id: drinkListView
  model: EdgeCoffeeMachine.drinksList
  delegate: Rectangle {
    // ...
    MouseArea {
      anchors.fill: parent
      onClicked: {
        console.log("Selected drink: " +
EdgeCoffeeMachine.drinksList.data(index).name)
     EdgeCoffeeMachine.selectBeverage(EdgeCoffeeMachine.drinksList.data(index))
      }
    }
  }
}
```

Manual user selection list binding

```
// views/UsersPanel.qml – horizontal user list bound to
usersList model
ListView {
  id: userList
  model: EdgeCoffeeMachine.usersList
  orientation: Qt.Horizontal
  delegate: userDelegate
}
```

Method invocation for drink reset/customization

```
// components/settings/SettingsPanel.qml – reset ingredients
from UI
MouseArea {
  anchors.fill: parent
  onClicked: {
    console.log("Reset settings to default for " +
root.targetBeverage.name)
    root.targetBeverage.resetIngredients()
  }
}
```

3.6.3 AI Component

- `recognize_user(frame: Image): integer`
  Method that executes the full face recognition pipeline using `ESP-DL`. It detects if a face is present and compares it against the enrolled database. It returns the `User ID` (positive integer) if the user is recognized, or `-1` if the face is not recognized or not present.
- `get_voice_command(stream: AudioStream): string`
  Method that executes a model to detect and recognize a voice command. It returns the string representing the recognized command.

3.7 Selected Architectural Styles and Patterns

The system's architecture is driven by stringent requirements, must be integrated directly into the board, and must be fully functional offline.

3.7.1 Modular Monolith

The application is packaged in a single executable (firmware) running on a single processor. However, it is logically decomposed into modules with distinct responsibilities (*UI*, *App logic*, and *AI*) and clear interfaces to ensure maintainability and parallel development.

3.7.2 Layered architecture

The software is structured in logical layers:

1. Presentation Layer (*UI/Frontend - Qt for MCU*)
2. Business Logic Layer (*App logic/Backend - C++/Qt, targeting ESP-IDF*)
3. Service Layer (*AI component - ESP-DL/ESP-SR*)

3.7.3 UI-Logic Decoupling

A cornerstone of this architecture is the strict decoupling between the Presentation Layer and the Business Logic Layer. This is not an optional

preference but a mandatory design choice that directly implements the required system architecture shown in the project's Component View (Figure 2).

Communication between these two distinct modules is exclusively managed via a formal API:

- **C++ Functions/Properties:**
  Exposed by the *Backend* that the *QML Frontend* can call to display data or trigger state changes;
- **Qt Signals:**
  Emitted by the *Backend* to notify the *Frontend* about state changes (e.g., when a drink is ready). The Frontend listens to these signals and updates the UI accordingly.

3.7.4 Edge AI

As a core principle of the project, no cloud architecture is used. All data processing (biometrics, voice) and AI model inference occur locally on the device to ensure privacy, low latency, and full offline capability.

This module is not a single library but a composite of two specialized Espressif frameworks:

- **ESP-DL**: For Face detection (FR-F-1) and Customer Recognition (FR-F-2)
  It is lightweight and general-purpose deep learning library (runtime) optimized for ESP32-P4. We'll execute a pretrained model on the device using the ESP-DL library.
- **ESP-SR**: For Voice Command (FR-V-1)
  To avoid the highly complex task of training a robust voice command model from scratch and accelerate development, we will adopt ESP-SR (Espressif Speech Recognition). This is a production-ready and highly-optimized framework dedicated specifically to voice applications on their chips. We will need to integrate it and configure it with the activation word and the list of commands specific to our project.

3.7.5 Recommendation Engine logic

The *Backend* implements a dynamic recommendation system based on an exponential decay model.
First element of this system is composed by how beverages are stored.

They are stored in a `WeightedSortedList` where the selected drink's weight increases while others decay, ensuring the list evolves with user habits. The second element is the user classification logic. The system tracks two scores, `tryerScore` (variety of drinks chosen) and `customizerScore` (frequency of ingredient modification), to classify users as:

- **Default**: Every user starts in this category. During this initial phase (first 3 interactions), the system prioritizes `Global Popularity` to sort the beverage list, while personal weights are silently initialized in the background.
- **Conservative**: The list is strictly sorted by habit/weight.
- **Early Adopter**: The sorting algorithm intentionally injects less frequent suggestions into top positions to encourage variety.

Each `Beverage` object manages its ingredients via `IngredientInfo` structures that enforce minimum, maximum, and default values in absolute physical units (grams for solids, milliliters for liquids), ensuring valid customization requests.

More details on the data model that supports this logic are provided in the next section.

3.8 Data Model Specification

This section details the system's data structures, including units of measurement, valid ranges, and validation rules.

3.8.1 Ingredients and Recipes Data Model

3.8.1.1 Units of Measurement

All ingredient quantities are stored and processed using absolute physical units:

Ingredient	Unit	Physical Meaning
Coffee Beans	grams (g)	Mass of ground coffee
Cocoa Powder	grams (g)	Mass of cocoa powder
Water	milliliters (ml)	Volume of water
Foam	milliliters (ml)	Volume of milk foam
Milk	milliliters (ml)	Volume of liquid milk

3.8.1.2 Valid Ranges and Defaults

Each ingredient has defined minimum and maximum bounds that reflect both physical constraints (hardware capacity, brewing quality) and recipe standards. These bounds depend on the specific beverage recipe.

Beverage	Coffee (g)	Cocoa (g)	Water (ml)	Foam (ml)	Milk (ml)
Espresso	10.0 [7–14]	N/A	30.0 [20–50]	N/A	N/A
Cappuccino	9.0 [7–14]	0.0 [0–5]	30.0 [20–50]	50.0 [20–80]	50.0 [0–100]
Americano	10.0 [7–14]	N/A	150.0 [100–250]	N/A	N/A
Latte	8.0 [7–14]	0.0 [0–5]	30.0 [20–50]	30.0 [0–60]	150.0 [50–250]
Mocha	10.0 [7–14]	10.0 [5–10]	50.0 [30–80]	40.0 [20–60]	100.0 [50–200]

Format: Default value [min–max] or N/A if the ingredient is not used in the recipe of the corresponsing beverage and its default, min and max are all 0.

Validation Rules:

1. **Range Enforcement**: For each ingredient, the constraint $\text{min} \leq \text{current} \leq \text{max}$ must always hold. Any user modification that violates this constraint is rejected at the UI level (sliders are bound to these ranges).
2. **Non-Negativity**: All ingredient quantities must be non-negative (≥ 0).
3. **Ingredient Availability**: Users can only modify ingredients that are part of the selected beverage recipe. If a beverage's recipe specifies $\text{max} = 0$ for a particular ingredient, that ingredient is not available for customization in the UI (FR-D-3).
4. **Default Value Validity**: The default value for each ingredient must satisfy $\text{min} \leq \text{default} \leq \text{max}$. This is enforced during beverage initialization in `RecipeDatabase`.

5. **Brewing Time Constraint**: The total brewing time calculated from all ingredient quantities must not exceed 180 seconds (NFR-6), which is implicitly guaranteed by the maximum ingredient limits.

3.8.2 User Profile Data Model

The `User` class maintains personalization data and behavioral metrics that power the adaptive recommendation engine. Each user maintains an independent weighted list of beverages that evolves based on their selections and customization behavior.

3.8.2.1 Core User Data Fields

Field	Type	Description
`id`	`int`	Unique identifier from facial recognition (AI embedding ID)
`name`	`std::string`	User's display name
`initials`	`std::string`	Derived initials (first and second character if space present)
`picture`	`int`	Profile picture index reference
`m_category`	`UserCategory`	Classification: Default, Conservative, or EarlyAdopter
`m_numBeverages`	`int`	Counter tracking total interactions (used for 3-interaction threshold)

3.8.2.2 Behavioral Scoring System

The recommendation engine uses two accumulated scores that update with each beverage brewing event:

Score	Range	Purpose
`m_tryerScore`	0.0 – 1.0	Measures tendency to select low-weight (novel) beverages; incremented for variety-seeking
`m_customizerScore`	0.0 – 1.0	Measures ingredient customization frequency; incremented when user modifies ingredients

Score	Range	Purpose
`m_customized`	`bool`	Session flag indicating ingredients were customized in current session

Score Update Mathematics:

When a beverage is brewed, the scores evolve using exponential decay. Let t be the `tryerScore` and c be the `customizerScore`. Then,

$$t_{\text{new}} = r_{\text{trier}} \times (1 - w) + (1 - r_{\text{trier}}) \times t_{\text{old}}$$

where:

- $r_{\text{trier}} = 0.25$ (learning rate for trying new drinks)
- w = current normalized weight of the selected beverage (0.0 = novel, 1.0 = favorite)

and

$$c_{\text{new}} = \begin{cases} r_{\text{customizer}} + (1 - r_{\text{customizer}}) \times c_{\text{old}} & \text{if customized} \\ (1 - r_{\text{customizer}}) \times c_{\text{old}} & \text{otherwise} \end{cases}$$

where $r_{\text{customizer}} = 0.225$ (learning rate for customization events).

3.8.2.3 User Classification Thresholds and Learning Rates

Users are classified into one of three categories based on their behavioral scores, which determines both the display order and the rate at which preferences evolve. Calling e the `earlyAdopterScore` (defined below), the classification rules are:

Category	Activation Condition	Weight Learning Rate r	Recommendation Behavior
Default	$n_{\text{beverages}} < 3$	$r_{\text{default}} = 0.175$	Display follows global popularity; personal weights silently initialized
Conservative	$n_{\text{beverages}} \geq 3$ AND $e < 0.5$	$r_{\text{cons.}} = 0.10$	Display strictly by personal weight (favorites first); slower adaptation

Category	Activation Condition	Weight Learning Rate r	Recommendation Behavior
Early Adopter	$n_{\text{beverages}} \geq 3$ AND $e \geq 0.5$	$r_{\text{earlyAd.}} = 0.25$	Display by personal weight with variety injection; faster adaptation

Early Adopter Score Calculation:

After 3 or more beverage selections ($n_{\text{beverages}} \geq 3$), users are reclassified using a combined `earlyAdopterScore` e defined as:

$$e = 0.8 \times t + 0.2 \times c$$

If this score exceeds the threshold of 0.5, the user becomes an Early Adopter; otherwise, they become Conservative. Once classified (at $n_{\text{beverages}} = 3$), reclassification occurs at each subsequent beverage selection, allowing users to transition between Conservative and Early Adopter states.

3.8.2.4 WeightedSortedList: Personal Beverage Preferences

Each user maintains a `WeightedSortedList<Beverage*>` that tracks normalized preference weights for every beverage. This list is the core data structure enabling the recommendation engine:

Component	Type	Description
`m_items`	`std::vector<Beverage*>`	Ordered list of beverage pointers, sorted by current weight (descending)
`m_weights`	`std::vector<float>`	Parallel array of normalized weights, always summing to 1.0
`m_weightR`	`float`	Learning rate (0.1–0.25) controlling weight update speed

Exponential Decay Weight Update Formula:

Each time a beverage is selected, the weights for the selected beverage and all other beverages are updated as follows:

$$W_{\text{selected}_{\text{new}}} = r + (1 - r) \times W_{\text{selected}_{\text{old}}}$$
$$W_{\text{other}_{\text{new}}} = (1 - r) \times W_{\text{other}_{\text{old}}}$$

where r is the learning rate (`m_weightR`). This ensures:

1. Selected beverage weight increases significantly
2. All other weights decay proportionally
3. Total weight sum remains exactly 1.0 (normalized invariant)
4. The selected item automatically "bubbles up" the sorted list

After each weight update, the internal insertion sort algorithm maintains descending order by weight, reordering items as needed to reflect the new weight distribution.

3.8.2.5 Display Beverage List Generation

The `m_displayBeverages` vector is the ordered list shown to the user in the UI, and its generation depends entirely on the user's classification:

Default User: Mirrors the global popularity ordering from `EdgeCoffeeMachine`, mapping global beverage names to the user's local cloned instances. Personal weights are accumulated in the background but not yet used for display.

Conservative User: Sorted strictly by personal weight in descending order. All beverages follow their weight ranking, with the highest-weighted at the top.

Early Adopter User: Sorted by personal weight (descending), but with a "variety injection" applied: if the list has 4 or more beverages, the lowest-weighted beverage (rarely selected) is removed from the last position and inserted at position 2 (index 2) to encourage exploration of lesser-known drinks.

This three-tier recommendation strategy ensures that:

- New users benefit from global consensus (popular drinks)
- Habitual users see their favorites prominently (familiar drinks)
- Adventurous users are gently nudged toward variety (novel drinks injected at visible position)

3.9 Other design decisions

In agreement with the customer the initial implementation of the system will **not** include a persistent storage. All data generated during the operative time of the system (user registration, user classifications, user preferences) will be held in RAM only. This means that when the device is rebooted, all data except the firmware will be lost.

This decision was made to prioritize the development of the core features, and is based on:

- The high intrinsic complexity of core features like the recommendation engine and the AI-driven interactions;
- The technical risk and uncertainty associated with the implementation of a persistent storage specifically for the ESP32-P4 board, for which the team does not currently have any prior experience.

3.10 Error Handling & Recovery

The system implements error handling for scenarios within the application domain. The following errors are defined, detected, and reported:

3.10.1 Handled Error Scenarios

1. Invalid Beverage Selection

- **Condition:** User attempts to brew a drink while no valid beverage is selected (e.g., `selectedBeverage` is `nullptr`).
- **Recovery:** The system logs an error message and updates the status to "Invalid beverage."
- **User Message:** "Invalid beverage." (displayed in status field)
- **User Action:** User must select a valid beverage from the list before attempting to brew.
- **Implementation:** Validated in `EdgeCoffeeMachine::makeDrink()`.

2. Brewing Already in Progress

- **Condition:** User attempts to initiate a new brew while another drink is being prepared.
- **Recovery:** The system rejects the request and maintains the current brewing state.
- **User Message:** "Already making a drink. Please wait." (displayed in status field)

- **User Action:** User must wait for the current brew to complete or cancel via the "Cancel" button.
- **Implementation:** Guarded by `isMakingDrink` flag in `EdgeCoffeeMachine::makeDrink()`.

3. User Not Found During Identification

- **Condition:** The AI subsystem detects a user ID that has not been previously enrolled in the system.
- **Recovery:** The system logs a warning but does not change user state; the machine remains in guest mode.
- **User Message:** No user-facing message; logged at debug level for system monitoring.
- **User Action:** User can enroll as a new user via the "Remember me" flow or continue as a guest.
- **Implementation:** Logged warning in `EdgeCoffeeMachine::identifyUser()`.

4. Duplicate User Enrollment

- **Condition:** The AI subsystem attempts to enroll a user ID that already exists in the system.
- **Recovery:** The system rejects the enrollment and logs the attempt.
- **User Message:** No user-facing message; silently prevented by the system.
- **User Action:** System does not allow re-enrollment of the same ID; user is redirected to login flow.
- **Implementation:** Guarded check in `EdgeCoffeeMachine::enrollUser()`.

3.10.2 Out-of-Scope Error Scenarios

The following error categories are **out of scope** for the current implementation, as they fall outside the application domain and are handled by hardware/firmware layers:

- **Sensor Faults:** Camera malfunction, microphone failure, or display driver errors are assumed to be handled by the ESP32-P4 board's firmware and platform layer. The application assumes sensors are functional and does not implement fallback logic.

- **Thermal Overshoot:** Heating element thermal runaway, pressure vessel failures, and other hardware safety issues are managed by the physical machine's firmware, not the application.
- **Ingredient Depletion:** The system does not monitor ingredient levels (no sensing hardware is available in the current scope). Operator responsibility to refill supplies.
- **Network Failures:** The application is designed for full offline operation; no cloud connectivity or synchronization is implemented.

3.11 Telemetry & Logging

The system implements structured logging to support development, debugging, and operational monitoring while respecting user privacy.

3.11.1 Logging Architecture

The application uses the `Qul::PlatformInterface::log()` function to emit log messages to the platform's logging subsystem. All logs are tagged with a component prefix (e.g., `[ECM]`, `[User]`) for easy filtering and analysis.

Logging Levels:

- **Info**: Normal operational events (user login, beverage selection, brew completion)
- **Warning**: Recoverable issues or unexpected state (empty beverage list, unrecognized user ID)
- **Error**: Application-level failures requiring investigation (invalid selections, logic errors)

3.11.2 Logged Events

Backend/Application Logic (`[ECM]`, `[User]`):

- Machine initialization and default beverage list loading
- User login/logout events (user name logged, not biometric data)
- Beverage selection and weight updates
- Brewing process start, completion, and cancellation
- User enrollment and identification attempts
- Classification transitions (Default → Conservative/Early Adopter)
- Recommendation score calculations (tryer score, customizer score, early adopter score)

3.11.3 Privacy & Data Protection

Logging adheres to the following privacy principles:

1. **Non-PII by Default:** User names are logged (as they are user-provided display names), but biometric data (facial embeddings, audio samples) are not logged; only high-level results (user ID or command) are recorded.
2. **Local-Only Logs:** All logs remain on the device in memory or platform-managed buffers. No logs are sent externally.
3. **Session Isolation:** Logs do not persist across device reboots. They are ephemeral and tied to the current session only.

4 User Interface Design

4.1 Home View

Figure 9: The coffee machine Home screen for users who have been identified.

When designing the user interface for the coffee machine, our primary goal was to minimize user interactions required to achieve the desired outcome of brewing a beverage. To accomplish this, several user actions were automated and handled by the machine's internal logic.

For example, instead of requiring users to manually initiate facial recognition, the system continuously scans for registered faces. As soon as a known user is detected, the Home View automatically updates with their personalized preferences. Because facial recognition is performed locally (on-premise), this design maintains high privacy and data security for all users.

Figure 10: The coffee machine Home screen for guest users.

To handle potential misidentifications, a "Not you?" button is displayed next to the welcome message. This allows quick access to the user list, where one can select a different profile, create a new account, or continue as a guest. Additionally, a "Logout" button is positioned in the top-right corner of the screen to end the current session and return to the guest view. If no user is recognized, the interface displays a generic welcome message, replacing these buttons with a "Remember me" option to enable the registration flow.

Both registered and guest users share a consistent layout. The central area highlights a featured beverage: either the preferred drink of the recognized user or the most popular one for guests. Next to the drink's image and name, a customization sidebar allows adjustments to parameters such as foam, coffee, milk, water, and powder levels depending on the drink type.

Beneath the customization section, the price and a prominent "Brew Now" button are displayed. On larger screens, an additional sidebar shows a list of available drinks, which can be customized according to the user's preferences.

Figure 11: The coffee machine Home screen for small screens.

Figure 12: Speech Recognition indicator.

To enhance interaction transparency during voice input, the bottom area of the interface features a sound wave animation and a real-time transcription of the user's spoken commands. Users can manually activate voice recognition by tapping the waveform. Once a beverage command is recognized, the featured drink updates automatically, and the user can confirm by pressing "Brew Now".

4.2 User Onboarding Views

When a new user signs up, the on boarding process consists of three steps:

1. **Name Input**: The user provides their name or username via voice input. The system confirms recognition accuracy, offering options to retry or continue.

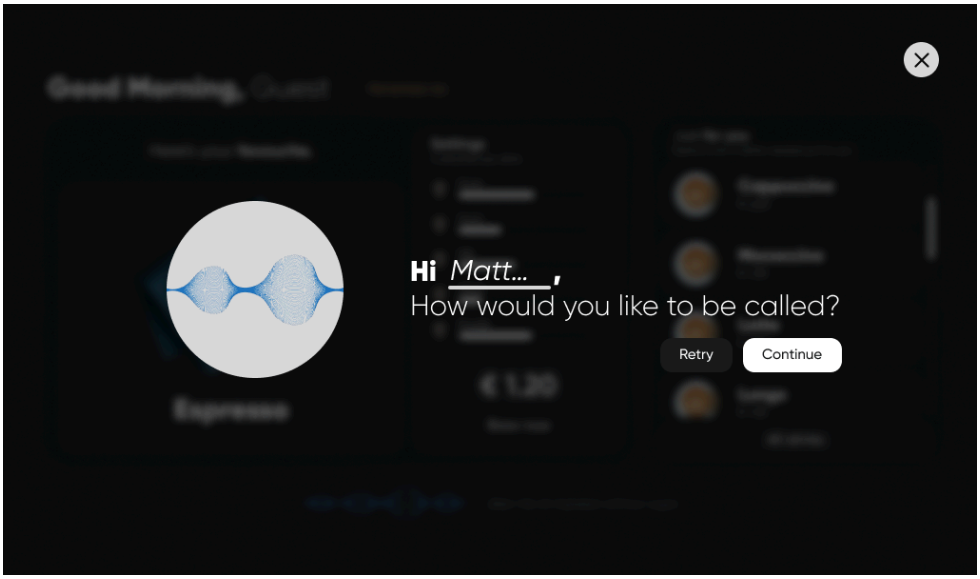


Figure 13: Onboarding: Name input

2. **Face Registration**: The system captures facial data and takes a profile picture. The user can retake or confirm the image before proceeding.

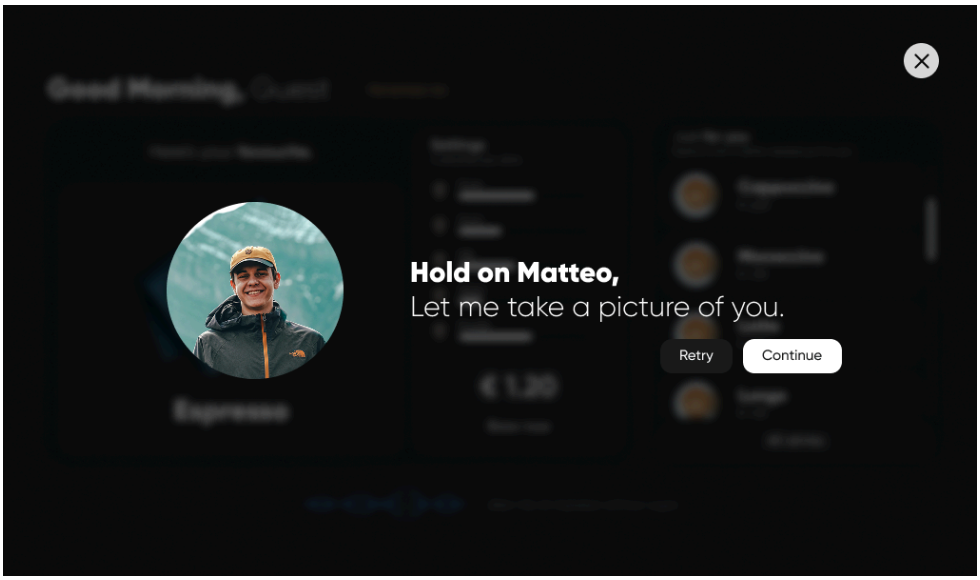


Figure 14: Onboarding: Face registration

3. **Welcome Screen**: The process concludes with a confirmation message, informing the user that they will now be automatically recognized by the machine in future sessions.

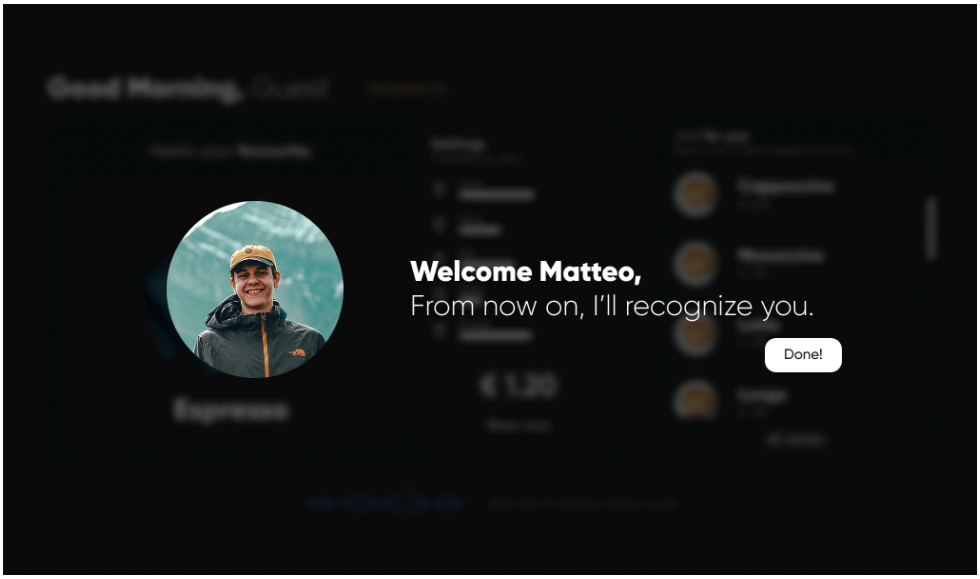


Figure 15: Onboarding: Welcome screen

4.3 Manual User Selection View

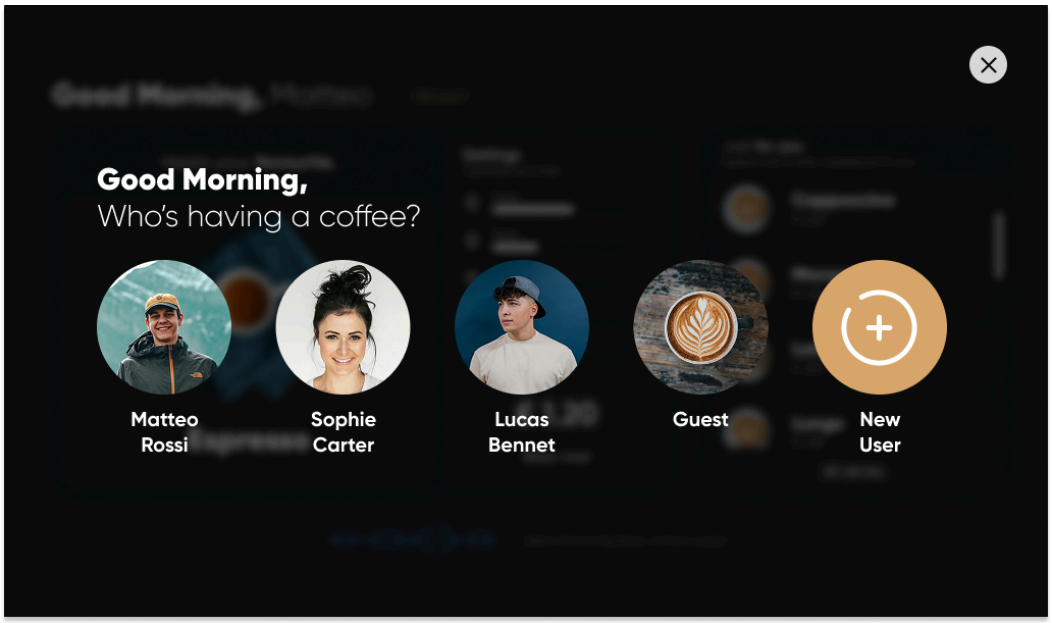


Figure 16: Manual user selection view

If the machine fails to identify a user, the **Manual User Selection View** allows them to choose a profile manually. This view displays all existing user profiles, along with a Guest option and the ability to add a new user.

4.4 Drinks List View

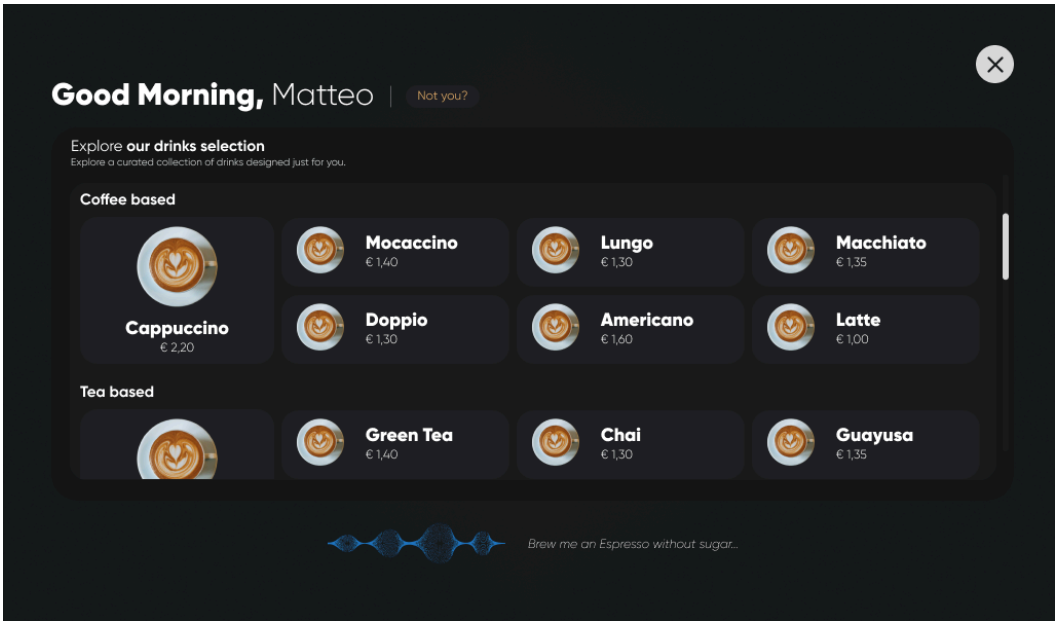


Figure 17: Drinks list view

On smaller displays, where space is limited, the full list of beverages is not shown on the **Home View**. By selecting the "All Drinks" button, users can browse the complete catalog of beverages, each accompanied by pricing information and grouped by drink category.

4.5 Brewing View

Figure 18: Home: Brewing view

When the brewing process begins, the user remains on the Home View. To draw focus, the rest of the interface is dimmed and slightly blurred, while the price component remains visible.

During brewing, the price area transforms into a progress indicator, displaying a cancel button and an animated wave that gradually rises as brewing progresses, reaching the top upon completion.

Figure 19: The color palette.

These designs will be iterated based on customer feedback throughout the development process and the feasibility of the implementation using the Qt Framework.

5 Implementation and Testing Plan

The project's initial phase was defined by a key technical challenge: the official Qt for MCU support package for the ESP32-P4-Function-EV board was not yet available. To mitigate this dependency, the team adopted a parallel development strategy, splitting the project into two independent tracks:

1. **Track 1: Desktop UI Application (UI + Qt-enabled Backend)** This track focused on developing the complete user experience on a standard desktop environment. It involved creating the QML-based user interface and integrating it with a version of the C++ backend that utilizes Qt's meta-object system. This allowed for rapid prototyping and testing of all UI-related features.
2. **Track 2: On-Board AI and Logic Integration (AI + Core Backend)** This track focused on the core embedded functionalities on the ESP32-P4 board. A version of the C++ backend logic, stripped of Qt dependencies, was integrated directly with the AI-driven face recognition application, proving the viability of the core on-device features.

Recently, the board support package has been provided by the customer. The team has therefore moved into the next phase: integrating the two development tracks into a single, unified application running on the target hardware. This integration process is currently underway and represents the main focus of the development effort, as the team works to resolve the complexities of merging the Qt-based UI with the on-board AI and logic components.

5.1 Testing Strategy Overview

The testing approach follows a three-layer pyramid: **unit tests** for core business logic, **UI integration tests** for QML/C++ interactions, and **hardware-in-the-loop (HIL) tests** for real-world device behavior. This strategy ensures comprehensive validation of all functional and non-functional requirements against acceptance criteria.

5.1.1 Unit Tests

The following unit tests are implemented in `tests/` using Catch2.

5.1.1.1 Beverage Model Tests

The `TestBeverage.cpp` file validates recipe data and ingredient customization logic:

- **Initialization**: Verifies all ingredients are correctly initialized with current, default, min, and max values.
- **Brewing Time Calculation**: Validates the formula `brewingTime() = sum(0.5 + normalized[i] * 10.0) * 1000 ms` across multiple ingredient configurations.
- **Reset Functionality**: Confirms `resetIngredients()` restores all values to defaults (FR-D-5).
- **Ingredient Bounds**: Ensures normalization respects min ≤ current ≤ max constraints (FR-D-3).

Acceptance Criteria: Each beverage must enforce ingredient ranges and calculate accurate brewing times ≤ 180 seconds (NFR-6).

5.1.1.2 User Classification & Recommendation Tests

The `TestUser.cpp` file validates user profile logic and personalization algorithms:

- **Default Category**: New users start in the "Default" category, shown global popularity list (FR-U-3).
- **Conservative Classification**: After ≥3 similar beverage selections with little customization, user is classified as "Conservative" and shown his favorites first (FR-U-4).
- **Early Adopter Classification**: High variety (different drinks) + high customization (ingredient modifications) → "EarlyAdopter" category; untried drink injected into top 3 recommendations (FR-U-5).
- **History Tracking**: `beverageBrewed()` increments drink counters; `beverageCustomized()` flags session for customization scoring (FR-S-1).
- **Error Resilience**: Null or unknown beverages handled gracefully without state corruption.

Acceptance Criteria: Classification must accurately reflect user behavior; recommendation list must be reordered correctly after each brew.

5.1.1.3 WeightedSortedList (Recommendation Algorithm) Tests

The `TestWeightedSortedList.cpp` file validates the exponential decay recommendation engine:

- **Uniform Initialization**: All items start with equal weight = 1.0 / count.
- **Weight Normalization**: Sum of all weights ≈ 1.0 (within float epsilon) after any operation.
- **Selection Boost**: When item is selected, its weight is $W = R + (1 - R) \times W_{\text{old}}$; others decay to $(1 - R) \times W_{\text{old}}$ (where R is the learning rate).
- **Reordering**: Items with higher weights automatically sort to the front of the list.
- **Configurable Learning Rate**: Different rates (0.2, 0.3, 0.5) produce expected weight deltas.
- **Edge Cases**: Empty lists handled safely.

Acceptance Criteria: Weights must remain normalized and sorted correctly across all operations. Learning rate must tune the speed of preference decay.

5.1.1.4 EdgeCoffeeMachine Core Logic Tests

The `TestEdgeCoffeeMachine.cpp` file validates the state machine and main controller:

- **Initialization**: Machine starts in "Idle" state with `isMakingDrink = false`, default beverage selected, no user logged in.
- **Beverage Selection**: `selectBeverage()` updates `selectedBeverage` property and `status` message (e.g., "Selected: Cappuccino").
- **Brewing State Transition**: `makeDrink()` sets `isMakingDrink = true` and updates status; `stopBrewing()` cancels and returns to idle.
- **User Session**: `setUser(user)` logs in; `logoutUser()` clears current user and reverts to guest menu (FR-UI-4).
- **Error Handling**: Attempting to brew without selection returns error status "Invalid beverage." (NFR-7).

Acceptance Criteria: All state transitions must be atomic and leave the machine in a consistent state.

5.1.2 UI Integration Tests

UI integration tests will validate the interaction between QML frontend and C++ backend on a desktop environment. Mock implementations will substitute the AI module (face recognition, voice commands) and brew timer for deterministic, repeatable testing. Since Qt for MCU does not provide UI testing support for the ESP32-P4-Function-EV board, a golden reference

approach will be used for these tests. Consequently, only backend changes that are reflected in the user interface will be validated.

5.1.2.1 Touch Navigation & Ingredient Customization

- **Drink List Binding**: QML ListView displays all beverages from `BeverageModel`; selecting a drink calls `EdgeCoffeeMachine::selectBeverage()`.
- **Ingredient Sliders**: Dragging a slider updates the corresponding `IngredientInfo::current` value; value is bounded by min/max.
- **Reset Button**: Tapping the reset button invokes `Beverage::resetIngredients()` (FR-D-5).

5.1.2.2 Recognition & Status Feedback

- **User Recognition Indication**: When a user is logged in, the UI displays their name and a welcome message (FR-UI-1).
- **Guest Mode Visual**: When in guest mode, the UI shows "Popular Drinks" and a "Remember me" button (FR-UI-1).

5.1.2.3 Brewing Display & Cancellation

- **Brew Screen Transition**: After `makeDrink()` is called, the UI transitions to a brewing screen (FR-UI-2).
- **Progress Animation**: A visual progress indicator (bar or waveform) animates during brewing; updates at least every 500 ms.
- **Cancel Button**: User can tap a cancel button during brewing, which calls `EdgeCoffeeMachine::stopBrewing()` (FR-UI-3).

5.1.2.4 Post-Brewing Reset

- **Idle Transition**: After brewing completes or is cancelled, the UI returns to the idle/recognition screen (FR-UI-4).
- **Auto-Logout**: The current user is logged out (`logoutUser()` called) after brewing finishes (FR-UI-4).
- **Ready for Next User**: System is ready to detect a new user immediately after; no stale state persists.

5.1.3 Hardware-in-the-Loop Tests

HIL tests run the complete firmware on the actual ESP32-P4-Function-EV board, validating real-time performance, AI inference, and device integration.

5.1.3.1 Facial Recognition Pipeline

- **Face Detection Latency**: On actual camera input, face detection completes within 2 seconds.
- **User Recognition**: Detected face is recognized and a valid user ID (≥ 1) is returned, or -1 if unknown (FR-F-2).
- **Confidence Threshold**: Faces with $< 60\%$ confidence are rejected; the system falls back to guest mode (FR-F-3).
- **Recognition Timeout**: Abort recognition after 10 seconds; gracefully return -1 (FR-F-4).
- **User Enrollment**: Unrecognized faces can be enrolled; a new `User` object is created with an assigned ID.

Performance Metrics:

- End-to-end recognition + menu display $\leq$ 10 seconds (NFR-1).
- Face detection $\leq$ 2 seconds.

5.1.3.2 Voice Command Pipeline

- **Command Recognition**: Spoken commands are detected and recognized; the system returns a command string (e.g., "make_espresso") (FR-V-1).
- **Confidence Threshold**: Commands with $< 60\%$ confidence are rejected; the UI shows "Didn't catch that" and offers retry (FR-V-2).
- **Audio Timeout**: Audio capture stops after 5 seconds of audio or 2 seconds of silence; the UI shows a retry prompt (FR-V-3).
- **Graceful Timeout**: Timeout does not crash the system; recovery to idle state completes within 5 seconds (NFR-7).

Performance Metrics:

- Voice command latency $\leq$ 3 seconds (from end of utterance to recognition result).

5.1.3.3 UI Responsiveness & Frame Rate

- **Touch Response**: Touch input registers within 100 ms; visual feedback (button press effect) appears within 100 ms (NFR-5).
- **Screen Transitions**: Menu updates and view transitions complete within 1500 ms (NFR-2).
- **Frame Rate**: UI renders at $\geq$ 20 FPS under nominal load (NFR-2).

5.1.3.4 End-to-End User Scenarios

Three complete user journeys are validated on hardware:

Scenario S1 – First-Time User:

1. User approaches machine; face is detected and enrolled.
2. User is shown default menu (FR-U-3); selects a drink.
3. User customizes ingredients (FR-D-2); selects within bounds (FR-D-3).
4. User issues voice command "Make espresso" (FR-V-1); command is recognized.
5. Brewing starts; progress displayed (FR-UI-2).
6. Brewing completes in ≤ 180 seconds (NFR-6).
7. System resets to idle within 5 seconds; ready for next user (FR-UI-4, NFR-7).

Scenario S2 – Returning Conservative User:

1. Recognized user's face triggers personalized menu (FR-F-2).
2. User is classified as "Conservative" (FR-U-4); top 2 favorites displayed first (FR-U-4).
3. User selects their favorite without customization.
4. Brewing starts via touch; completes; system resets.

Scenario S3 – Early Adopter with Customization & Cancellation:

1. Recognized user shown "EarlyAdopter" menu with novel drinks injected (FR-U-5).
2. User selects an untried drink and customizes ingredients heavily (FR-D-2).
3. User cancels mid-brew by tapping cancel button (FR-UI-3).
4. System stops brewing and returns to idle within 5 seconds (NFR-7).

5.1.4 Requirements Mapping

The highlights below provide traceability from each requirement to its corresponding unit, integration, or HIL test.

- **FR-D-1 to FR-D-5** (Drink Catalogue & Customization): Unit tests validate; integration & HIL tests validate UI binding.
- **FR-U-2 to FR-U-5** (User Classification & Menus): Unit tests validate classification logic; HIL tests validate real-world recognition.
- **FR-F-1 to FR-F-4** (Face Detection & Recognition): HIL tests validate; unit tests mock AI responses.

- **FR-V-1 to FR-V-3** (Voice Commands): HIL tests validate; integration tests mock voice results.
- **FR-T-1** (Touch Navigation): Integration tests validate QML binding; HIL tests validate hardware responsiveness.
- **FR-UI-1 to FR-UI-4** (UI Feedback & Lifecycle): Integration tests validate property binding; HIL tests validate real-time display.
- **NFR-1 to NFR-9** (Performance, Privacy, Reliability): HIL tests measure real-world timing; integration tests validate under mock load.

6 Glossary

6.1 Acronyms

- **ECM**: Edge Coffee Machine.
- **UI**: User Interface.
- **UX**: User Experience.
- **MCU**: Microcontroller Unit.
- **AI**: Artificial Intelligence.
- **HIL**: Hardware-in-the-Loop.
- **FPS**: Frames Per Second.
- **RASD**: Requirements Analysis and Specification Document.
- **GUI**: Graphical User Interface

Design References

[1] M. Deshmir, H. Mahdizadehm, M. Kazemkhani, and M. Askarzade, "Allespresso, Coffee Maker UI/UX Design." [Online]. Available: https://www.behance.net/gallery/206658869/Allespresso-Coffee-Maker-UIUX-Design

[2] M. Rahimi, "Delivery coffee application." [Online]. Available: https://dribbble.com/shots/20353652-Delivery-coffee-application

[3] A. Masud, "Coffee App Design – UI UX." [Online]. Available: https://dribbble.com/shots/25873648-Coffee-App-Design-UI-UX

[4] T. Moeller, "Coffee App." [Online]. Available: https://dribbble.com/shots/21273795-Coffee-App

[5] S. Windmill, "Smart coffee machine app." [Online]. Available: https://dribbble.com/shots/23534133-Smart-coffee-machine-app